Graph Modeling

When the graph is not in the input

Learning goals

- Turn a story into a graph: choose **nodes = states**, **edges = moves**
- Augment states with extra information (coupon used, fuel, keys, time mod k)
- Pick the cheapest correct search: BFS, multi-source BFS, 0-1 BFS, Dijkstra
- Use virtual nodes and reversed edges to shrink graphs
- Estimate state-space size before coding

The modeling question

> "If I froze the process at one moment, what would I need to write down to continue it?"

- That snapshot is a **state** → a node
- Each legal action is an **edge**, with a cost
- The answer is a shortest path / reachability query on this graph
- Count states × edges per state → is it fast enough?

Choosing the search

Edge costs	Algorithm	Time
all equal	BFS	O(V + E)
several sources, equal	multi-source BFS	O(V + E)
0 or 1	0-1 BFS (deque)	O(V + E)
small integers ≤ K	split edges or dial's buckets	O(K·V + E)
non-negative	Dijkstra	O(E log V)
negative	Bellman–Ford / SPFA (rare)	O(VE)

Pattern 1: layered states

CSES 1195 Flight Discount: one flight may be taken at half price.

```
state (city, used)          used ∈ {0, 1}
edge u→v, cost c:
  (u,0) → (v,0)  cost c
  (u,0) → (v,1)  cost ⌊c/2⌋
  (u,1) → (v,1)  cost c
```

- Two copies of the graph, connected by "coupon" edges
- Generalises to k coupons: k+1 layers

Dijkstra on states: code

```
vector<array<ll,2>> dist(n + 1, {INF, INF});
priority_queue<tuple<ll,int,int>, vector<tuple<ll,int,int>>, greater<>> pq;
dist[1][0] = 0; pq.push({0, 1, 0});
while (!pq.empty()) {
    auto [d, u, used] = pq.top(); pq.pop();
    if (d != dist[u][used]) continue;
    for (auto [v, c] : adj[u]) {
        if (d + c < dist[v][used])
            { dist[v][used] = d + c; pq.push({d + c, v, used}); }
        if (!used && d + c/2 < dist[v][1])
            { dist[v][1] = d + c/2; pq.push({d + c/2, v, 1}); }
    }
}
```

Pattern 2: resource in the state

UVa 11367 Full Tank?: cities sell fuel at different prices; tank capacity ≤ 100.

```
state (city, fuel)
  buy one unit:   (u, f)   → (u, f+1)       cost price[u]
  drive edge d:   (u, f)   → (v, f−d)       cost 0   (if f ≥ d)
```

- 1000 cities × 101 fuel levels ≈ 10^5 states: Dijkstra per query
- "Buy one unit at a time" keeps the out-degree small

Pattern 3: bitmask in the state

Grid with keys `a–f` and doors `A–F`:

```
state (row, col, keysMask)     R·C·2⁶ states
move to door X: allowed iff mask has bit X
move to key x:  mask |= bit x
```

- BFS, `dist` array indexed `(r*C + c)*64 + mask`
- Similar: "visited these k special cells" (≤ 16 → TSP-like BFS)

Pattern 4: time / phase in the state

- Traffic lights with period p, monsters moving cyclically, tides
- State `(cell, t mod p)`; waiting = edge to `(cell, t+1 mod p)`
- Dijkstra with "earliest arrival": if edges are only usable at certain times, waiting is optimal to delay *after* arrival (FIFO property)

Pattern 5: direction in the state

- "Turning costs 1, going straight is free" → state `(cell, dir)`
- Straight move: weight 0; turn: weight 1 → **0-1 BFS**
- Same: mirrors and lasers, minimum number of turns of a robot

```
(r,c,d) —0—▶ (r+dr[d], c+dc[d], d)
(r,c,d) —1—▶ (r, c, (d+1)%4)    and  (r, c, (d+3)%4)
```

0-1 BFS

- Deque: weight-0 edge → `push_front`, weight-1 edge → `push_back`
- Deque stays sorted with two distinct distance values
- Like Dijkstra: a node can be pushed more than once; relax with `<`

```
if (d + w < dist[v]) {
    dist[v] = d + w;
    if (w == 0) dq.push_front(v); else dq.push_back(v);
}
```

Example: Wizard in Maze

AtCoder ABC 176 D: walk to a neighbour for free, or "warp" to any road cell in the surrounding 5×5 square for 1 magic. Minimise magic.

```
for (int k = 0; k < 4; k++) {           // walk: weight 0
    ... if (d < dist[nr][nc]) { dist[nr][nc] = d; dq.push_front({nr,nc}); }
}
for (int i = -2; i <= 2; i++)           // warp: weight 1
  for (int j = -2; j <= 2; j++) {
    ... if (d + 1 < dist[nr][nc]) { dist[nr][nc] = d + 1; dq.push_back({nr,nc}); }
  }
```

- 10^6 cells × 29 edges: fine with 0-1 BFS, slow with Dijkstra's log

Pattern 6: multi-source BFS

- "Distance to the nearest X" for every cell: push **all** X's at distance 0
- One BFS instead of one per source
- *UVa 11624 Fire!*: several fires spread; Joe escapes if he reaches a cell strictly before the fire

```
1) multi-source BFS from all 'F' → fireTime[cell]
2) BFS for Joe; enter a cell only if joeTime + 1 < fireTime
```

- CSES 1194 *Monsters* is the same with path output

Pattern 7: reversed edges

- "For each node, distance to the nearest node with property P"
- Edges `i → j`; reversing them turns "from each i" into "from all P's"
- *CF 1272E Nearest Opposite Parity*: jumps `i → i ± a_i`; reverse edges, multi-source BFS from all even-valued and all odd-valued positions

Pattern 8: virtual nodes

- Many-to-many edges (every portal reaches every other) = $O(k^2)$ edges
- Add a hub node: `portal → hub` cost x, `hub → portal` cost 0 → O(k)
- Group edges: "all stations on line L are connected" → node per line
- *CF 1520G To Go Or Not To Go?*: portals with costs; trick: BFS from start and from end, combine through the best portal

Estimate before coding

Model	States	Edges / state	OK?
(cell, mask) $1000^2 \times 2^6$	$6.4 \cdot 10^7$	4	BFS, tight memory
(city, fuel) 1000×100	10^5	~10	Dijkstra ✓
(cell, dir) $10^6 \times 4$	$4 \cdot 10^6$	3	0-1 BFS ✓
(city, time) $10^5 \times 10^4$	10^9	—	× rethink

- Memory: `int dist[6.4e7]` = 256 MB — use `char` / `short` or bitsets

Correctness traps

- Incomplete state (forgot the direction / key mask / parity)
- Marking visited at **pop** in BFS → duplicates; at **push** is standard
- Dijkstra with negative edges hidden in the model
- Checking the goal only in one layer (`min(dist[n][0], dist[n][1])`!)
- Multi-source BFS with ties: strict vs. non-strict comparisons

Complexity summary

- BFS / 0-1 BFS: $O(S + T)$ for S states and T transitions
- Dijkstra: $O(T \log S)$
- Augmenting the state by a factor k multiplies both by k
- If $S \cdot \log S$ is $\sim 10^8$, prefer 0-1 BFS or bucket queues when costs are small

A real regional flavour

Kattis `getshorty` *(Get Shorty)*: each corridor keeps a fraction f ≤ 1 of your size; maximise the final size.

- Product of factors → maximise ⇒ Dijkstra with `max` and multiplication
- Works because factors ≤ 1 (monotone, like non-negative weights)
- Alternative: weights `−log f ≥ 0` and standard Dijkstra

Practice problems

- CSES 1195 — Flight Discount; CSES 1194 — Monsters
- CSES 1202 — Investigation (counting shortest paths on top of Dijkstra)
- AtCoder ABC 176 D — Wizard in Maze
- UVa 11624 — Fire!; UVa 11367 — Full Tank?
- Codeforces 1063B — Labyrinth (0-1 BFS with left/right limits)
- Codeforces 1272E — Nearest Opposite Parity
- Codeforces 1520G — To Go Or Not To Go?
- Codeforces 590C — Three States (BFS from each of three regions)
- Kattis `getshorty`

Summary

- State = everything needed to continue; edges = actions
- Edge weights decide the algorithm: BFS / 0-1 BFS / Dijkstra
- Multi-source and reversed edges answer "nearest X" for all nodes at once
- Virtual nodes turn quadratic edge sets linear
- Next: regional DP patterns